

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 04 -

MELHORANDO A API: FASTAPI E RECURSOS AVANÇADOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	29
REFERÊNCIAS.....	30

EMAND

O QUE VEM POR AÍ?

Agora que conhecemos o Flask, vamos avançar para o FastAPI, um framework moderno, rápido e muito amigável para APIs de Machine Learning.

Ao longo da aula você também verá como o FastAPI ficou tão popular para APIs de dados/ML e quais são as diferenças principais em relação ao Flask. O processo de criação de um aplicativo FastAPI envolve a definição de endpoints (usando `@app.get`, `@app.post`), o uso de modelos de dados com Pydantic para validação automática e a geração automática de documentação (Swagger UI, acessível em `/docs`). É possível realizar a reimplementação da rota `/predict` utilizando o FastAPI.

Também serão abordadas noções de assincronicidade (`async def`) e quando isso é útil, além de uma visão rápida de middlewares e dependency injection para carregar modelos apenas uma vez.

HANDS ON

Prepare-se para mais uma aula hands on, na qual veremos na prática a evolução da nossa API para um framework mais robusto e completo. Abram o VSCode e assistam os vídeos para a parte prática!



SAIBA MAIS

Por que FastAPI

Por que evoluir de Flask para FastAPI? Na aula anterior implementamos uma API simples em Flask para expor um modelo de ML. Isso funcionou, mas **esbarramos em limitações importantes**. O Flask é minimalista, não oferecendo suporte nativo a requisições assíncronas, nem validação de dados ou documentação automática, fator este determinante para que eu pessoalmente sempre utilize FastAPI mesmo em projetos mais simples.



Figura 1 – FastAPI
Fonte: Google Imagens (2026)

O FastAPI é um framework web moderno para Python, lançado em 2018 por Sebastián Ramírez. Ele rapidamente ganhou tração por combinar alta performance e facilidade de uso. O FastAPI, em sua essência técnica, não é uma criação totalmente nova. Ele se baseia em elementos já estabelecidos: usa o Starlette, um micro-framework ASGI conhecido por sua alta velocidade, e o Pydantic, fundamental para a validação e modelagem de dados.

Ele também utiliza type hints do Python para gerar validação e documentação automática e, sem esforço extra, gera uma interface de docs interativa nas rotas /docs (Swagger UI) e /redoc(ReDoc). Para ML isso é ouro, pois você descreve claramente o contrato da API (inputs/outputs) e o time inteiro ou até mesmo outros serviços conseguem experimentar a API facilmente via Swagger. Os erros também são capturados automaticamente, como por exemplo no caso de campo ausente, tipo errado etc.

Hands On - Criando Nosso Primeiro App FastAPI

Vamos começar criando um app bem simples, estilo “Hello, FastAPI”.

```
aula 04 > hello_fastapi.py > ...
1  from fastapi import FastAPI
2
3
4  app = FastAPI(title="API Básica com FastAPI")
5
6
7  @app.get("/")
8  def read_root():
9      return {"message": "Hello, FastAPI!"}
10
```

Figura 2 –Hello, FastAPI(1)
Fonte: Elaborado pelo autor (2026)

Rodando o Servidor com Uvicorn

O FastAPI em si é só o framework. Para rodar o servidor HTTP usamos um ASGI server, como o Uvicorn.

Passo 1 - Instale as dependências no terminal:

```
pip install fastapi uvicorn
```

Passo 2 - Rode o servidor apontando o objeto ‘app’ do arquivo que criamos:

```
PS C:\B3-API\aula 04> python -m uvicorn hello_fastapi:app --reload
INFO: Will watch for changes in these directories: ['C:\\B3-API\\aula 04']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [18456] using WatchFiles
INFO: Started server process [15672]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figura 3 –Hello, FastAPI(2)
Fonte: Elaborado pelo autor (2026)

Passo 3 - Abra o navegador em:

- http://127.0.0.1:8000/ → deve ver o JSON
{"message": "Hello, FastAPI!"}

- http://127.0.0.1:8000/docs → Swagger UI gerado automaticamente
- http://127.0.0.1:8000/redoc → documentação ReDoc

Repare que, sem nenhuma configuração extra, você já ganhou uma UI de documentação fantástica.

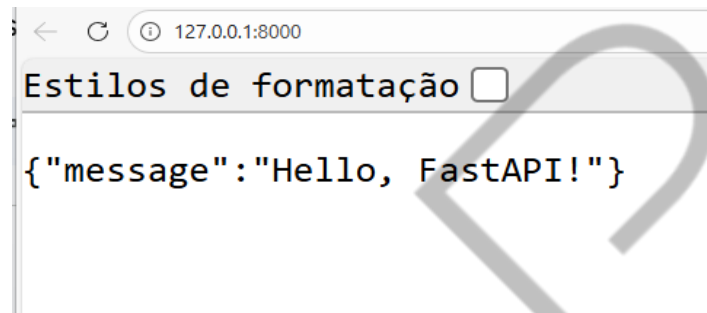


Figura 4 – Fastapi - Json
Fonte: Elaborado pelo autor (2026)

Com o Swagger podemos testar a API diretamente, algo que com o Flask só seria possível instalando o Flasgger.

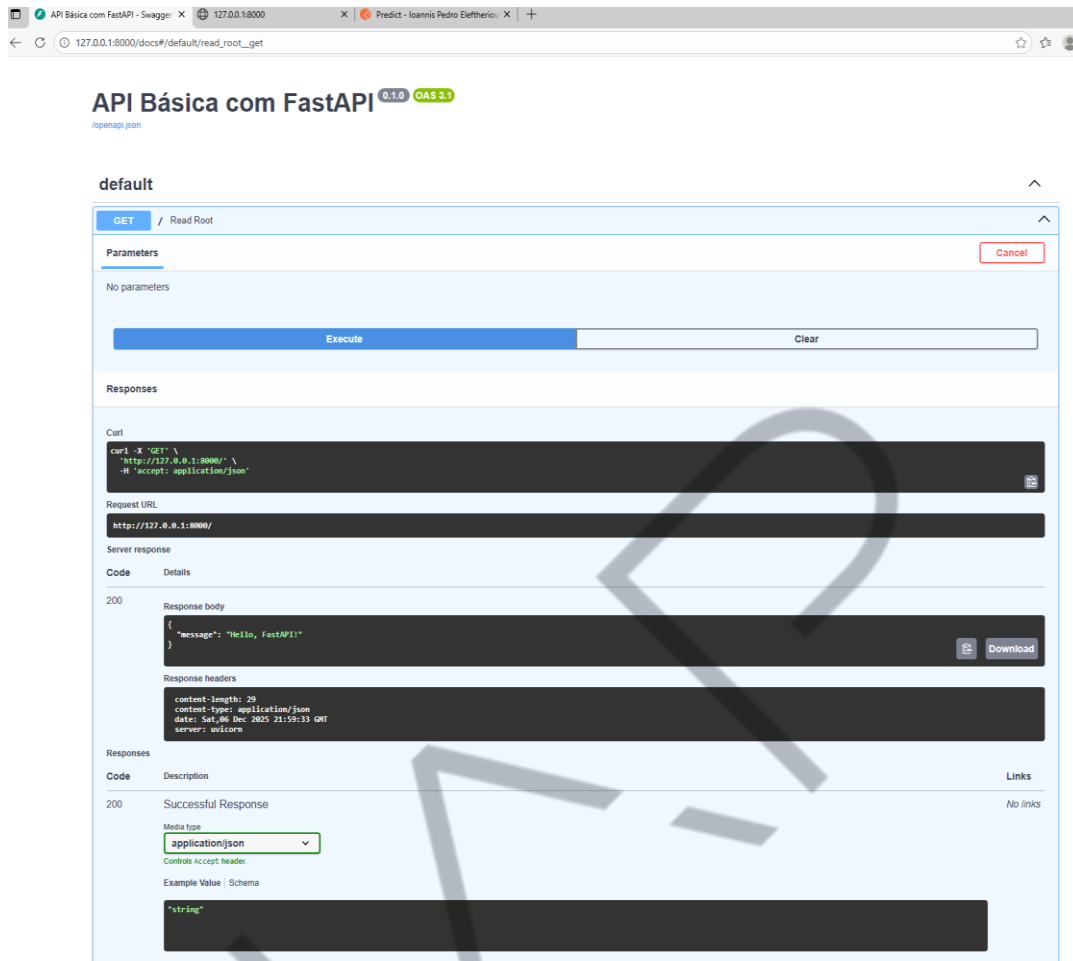


Figura 5 – Swagger - Hello, FastAPI
Fonte: Elaborado pelo autor (2026)

O ReDoc, apesar de ser uma interface apenas leitura, não permitindo testes, possui um layout em três colunas estilo documentação pronta para o consumidor final, muito mais elegante e clean. Resumindo:

- Swagger = para você testar a API.
- ReDoc = para você compartilhar a documentação bonita com outros times.

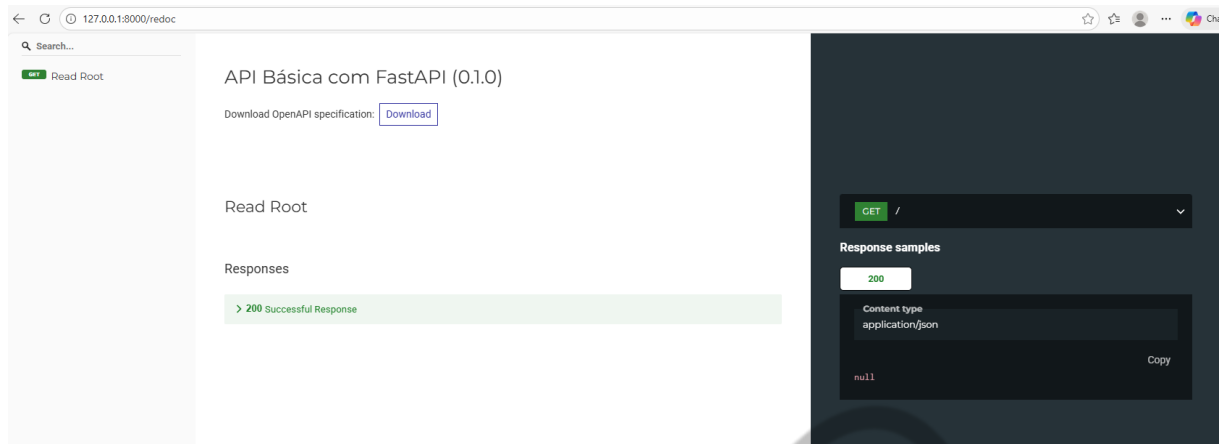


Figura 6 – ReDoc- Hello, FastAPI
Fonte: Elaborado pelo autor (2026)

Modelos de dados com Pydantic

Uma das maiores melhorias do FastAPI em relação ao Flask é o uso de Pydantic para modelar e validar dados. No Flask, recebíamos um `request.json` e precisávamos manualmente extrair campos, verificar tipos, tratar ausências etc. Com FastAPI, definimos classes que herdam de `pydantic.BaseModel` representando o esquema dos dados esperados e o framework faz a magia.

RECURSO	DESCRIÇÃO
Validação de tipos	Garante que cada campo tem o tipo correto
Valores padrão	campo: float = 0.0
Validação customizada	Field(ge=0, le=10) para ranges
Descrições	Field(description="...") para docs
Exemplos	Field(example=5.1) aparecem no Swagger

Tabela 1 - Vantagens do Pydantic
Fonte: Elaborado pelo autor (2026)

Agora vamos reimplementar a API de classificação Iris usando o modelo que treinamos na aula anterior!

IMPORTS	PARA QUE SERVE
FastAPI	Criar a aplicação web
HTTPException	Retornar erros HTTP (ex: 503)
BaseModel, Field	Definir e validar estrutura dos dados
pickle	Carregar o modelo .pkl do disco
numpy	Preparar os dados para o modelo

Tabela 2 - Reimplementando a API de classificação Iris
Fonte: Elaborado pelo autor (2026)

Depois de instaladas e importadas as bibliotecas, vamos criar as classes que descrevem a estrutura dos dados que a API recebe e retorna. Se alguém enviar “texto” onde deveria ser número, graças ao Pydantic, o FastAPI retorna erro 422 automaticamente. O melhor é que toda essa documentação vai aparecer de maneira automatizada no Swagger/Redoc, além de ajudar também a IDE.

```
26 class IrisRequest(BaseModel):
27     """
28     Modelo de entrada para requisicoes de predicao.
29
30     Representa as 4 medidas de uma flor Iris que serao usadas
31     para classificar a especie.
32
33     Attributes:
34         sepal_length: Comprimento da sepala em centimetros
35         sepal_width: Largura da sepala em centimetros
36         petal_length: Comprimento da petala em centimetros
37         petal_width: Largura da petala em centimetros
38     """
39
40     sepal_length: float = Field(
41         ...,
42         description="Comprimento da sepala (cm)",
43         example=5.1
44     )
45     sepal_width: float = Field(
46         ...,
47         description="Largura da sepala (cm)",
48         example=3.5
49     )
50     petal_length: float = Field(
51         ...,
52         description="Comprimento da petala (cm)",
53         example=1.4
54     )
55     petal_width: float = Field(
56         (function) example: Any petala (cm)",
57         example=0.2
58     )
59
60
```

Figura 7 – FastAPI-IRIS-1
Fonte: Elaborado pelo autor (2026)

```
class IrisPrediction(BaseModel):
    """
    Resultado da classificacao.

    Attributes:
        classe: Nome da especie prevista (setosa, versicolor ou virginica)
        classe_idx: Indice numerico da classe (0, 1 ou 2)
    """

    classe: str = Field(..., description="Nome da especie prevista")
    classe_idx: int = Field(..., description="Indice numerico da classe")


class IrisProbabilities(BaseModel):
    """
    Probabilidades de cada classe.

    Representa a confianca do modelo para cada especie.
    Os valores somam 1.0 (100%).

    Attributes:
        setosa: Probabilidade de ser Iris Setosa
        versicolor: Probabilidade de ser Iris Versicolor
        virginica: Probabilidade de ser Iris Virginica
    """

    setosa: float
    versicolor: float
    virginica: float
```

Figura 8 – FastAPI-IRIS-2
Fonte: Elaborado pelo autor (2026)

```
class IrisResponse(BaseModel):
    """
    Resposta completa da API para predicoes.

    Attributes:
        sucesso: Indica se a predicao foi realizada com sucesso
        predicao: Objeto com a classe prevista
        probabilidades: Objeto com as probabilidades de cada classe
        entrada_recebida: Echo dos dados enviados pelo cliente
    """

    sucesso: bool
    predicao: IrisPrediction
    probabilidades: IrisProbabilities
    entrada_recebida: IrisRequest
```

Figura 9 – FastAPI-IRIS-3
Fonte: Elaborado pelo autor (2026)

Agora, vamos carregar o modelo treinado do disco para a memória e criar a instância principal da aplicação.

```
try:
    with open('modelo_iris.pkl', 'rb') as f:
        modelo = pickle.load(f)
    with open('classes_iris.pkl', 'rb') as f:
        classes = pickle.load(f)
    MODELO_CARREGADO = True
except FileNotFoundError:
    modelo = None
    classes = None
    MODELO_CARREGADO = False

app = FastAPI(
    title="API de Classificacao Iris",
    description="API para classificar especies de flores Iris usando Machine Learning.",
    version="2.0.0",
)
```

Figura 10 – FastAPI-IRIS-4
Fonte: Elaborado pelo autor (2026)

Começamos pelo endpoint GET /, rota principal que retorna as informações sobre a API. É útil para verificar se ela está no ar e ver os endpoints disponíveis.

```
@app.get("/")
def home():
    """
    Retorna informacoes gerais sobre a API.

    Returns:
        dict: Nome, versao, descricao e endpoints disponiveis
    """
    return {
        "nome": "API de Classificacao Iris",
        "versao": "2.0 (FastAPI)",
        "descricao": "API para classificar especies de flores Iris",
        "endpoints": {
            "GET /": "Informacoes da API",
            "GET /health": "Status de saude",
            "GET /docs": "Documentacao interativa (Swagger)",
            "POST /predict": "Fazer previsao"
        },
        "modelo_carregado": MODELO_CARREGADO
    }
```

Figura 11 – FastAPI-IRIS-5
Fonte: Elaborado pelo autor (2026)

Agora temos a rota GET /health, para verificarmos em produção o monitoramento de se a API está funcionando corretamente.

```
@app.get("/health")
def health():
    """
    Verifica o status de saude da API.

    Endpoint usado para monitoramento e health checks
    em ambientes de producao (Docker, Kubernetes, etc).

    Returns:
    | dict: Status da API e estado do modelo
    """
    return {
        "status": "healthy" if MODELO_CARREGADO else "unhealthy",
        "modelo_carregado": MODELO_CARREGADO
    }
```

Figura 12 – FastAPI-IRIS-6
Fonte: Elaborado pelo autor (2026)

Agora, para a rota POST /predict, vamos seguir quatro passos:

- Verificar se o modelo está carregado.
- Preparar os dados para o modelo.
- Fazer a predição.
- Montar a resposta usando os modelos Pydantic.

```
@app.post("/predict", response_model=IrisResponse)
def predict(payload: IrisRequest) -> IrisResponse:
    """
    Realiza a classificacao de uma flor Iris.

    Recebe as 4 medidas da flor e retorna a especie prevista
    junto com as probabilidades de cada classe.

    Args:
    |   payload: Objeto IrisRequest com as medidas da flor

    Returns:
    |   IrisResponse: Predicao, probabilidades e dados de entrada

    Raises:
    |   HTTPException: 503 se o modelo nao estiver carregado
    """
    if not MODELO_CARREGADO:
        raise HTTPException(
            status_code=503,
            detail="Modelo nao carregado. Verifique se os arquivos .pkl existem."
        )

    features = np.array([
        payload.sepal_length,
        payload.sepal_width,
        payload.petal_length,
        payload.petal_width
    ])

    predicacao_idx = modelo.predict(features)[0]
    probabilidades = modelo.predict_proba(features)[0]

    return IrisResponse(
        sucesso=True,
        predicacao=IrisPrediction(
            classe=classes[predicacao_idx],
            classe_idx=int(predicacao_idx)
        ),
        probabilidades=IrisProbabilities(
            setosa=round(float(probabilidades[0]), 4),
            versicolor=round(float(probabilidades[1]), 4),
            virginica=round(float(probabilidades[2]), 4)
        ),
        entrada_recebida=payload
    )
```

Figura 13 – FastAPI-IRIS-7
Fonte: Elaborado pelo autor (2026)

Testando a API

Vamos iniciar o servidor. No terminal, navegue até a pasta da aula e execute o seguinte: `python -m uvicorn app_fastapi_iris:app --reload`

Agora, abra no navegador: `http://127.0.0.1:8000/docs`. Você verá o Swagger UI com a descrição da API que preenchemos no FastAPI, todos os endpoints

documentados, os Schemas dos modelos Pydantic e um botão “Try it out” para testar cada endpoint.

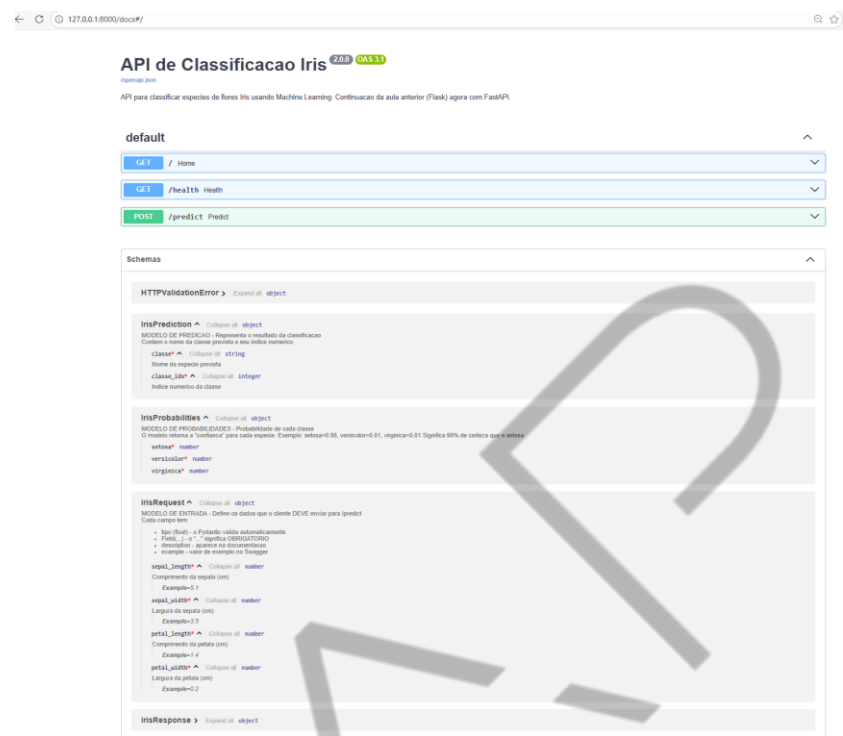


Figura 14 – FastAPI-IRIS-8
Fonte: Elaborado pelo autor (2026)

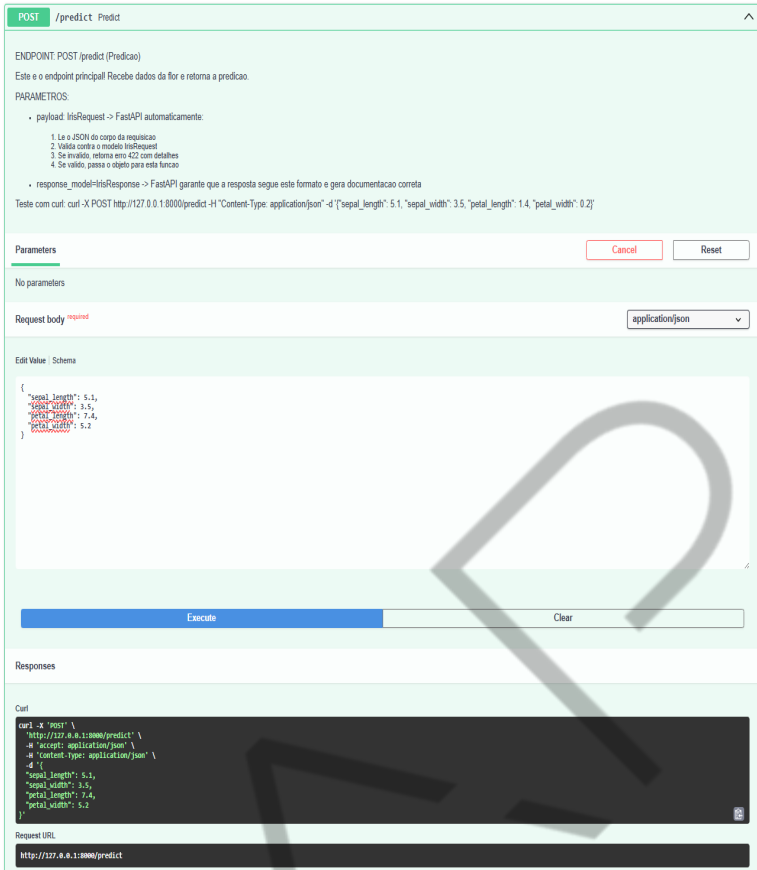


Figura 15 – FastAPI-IRIS-9
Fonte: Elaborado pelo autor (2026)

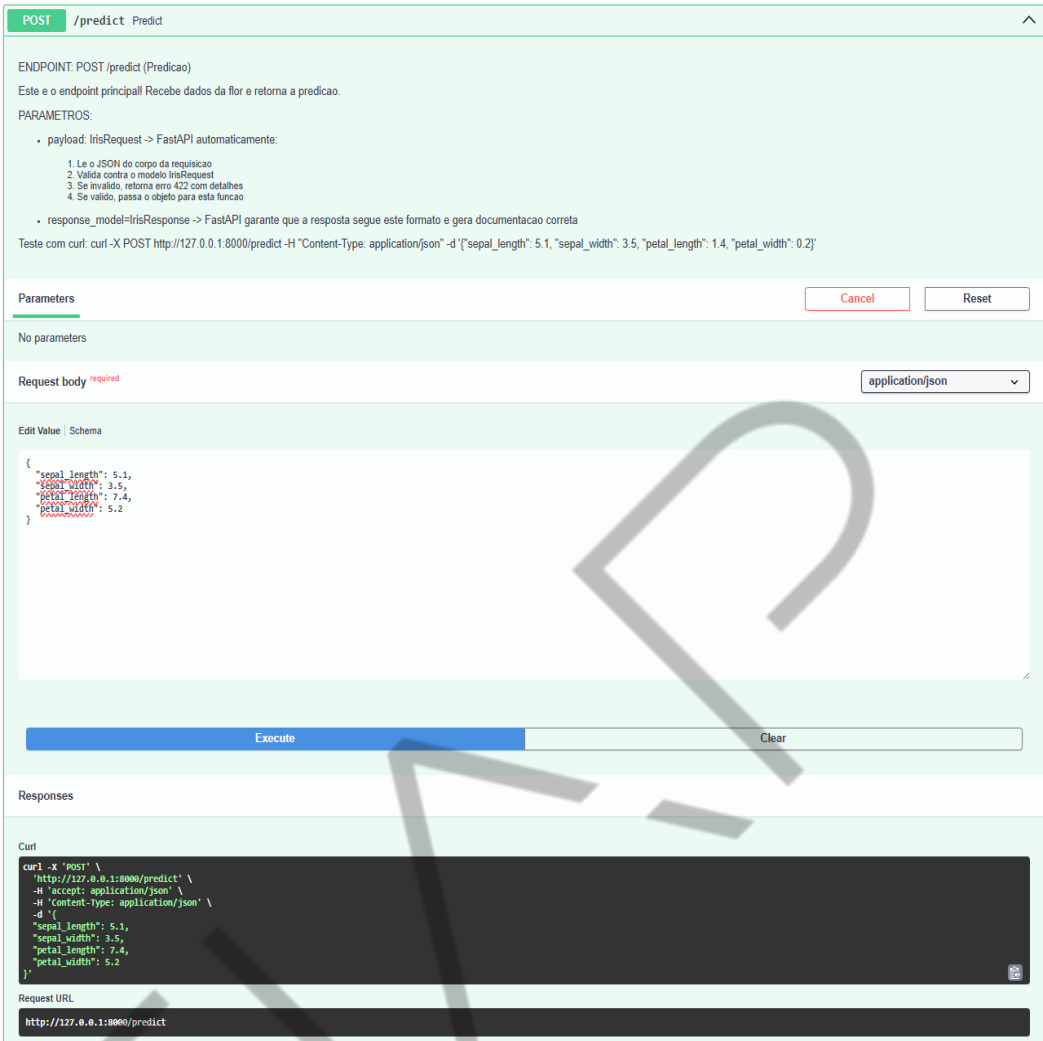


Figura 16 – FastAPI-IRIS-10
Fonte: Elaborado pelo autor (2026)

Retorno com a predição:

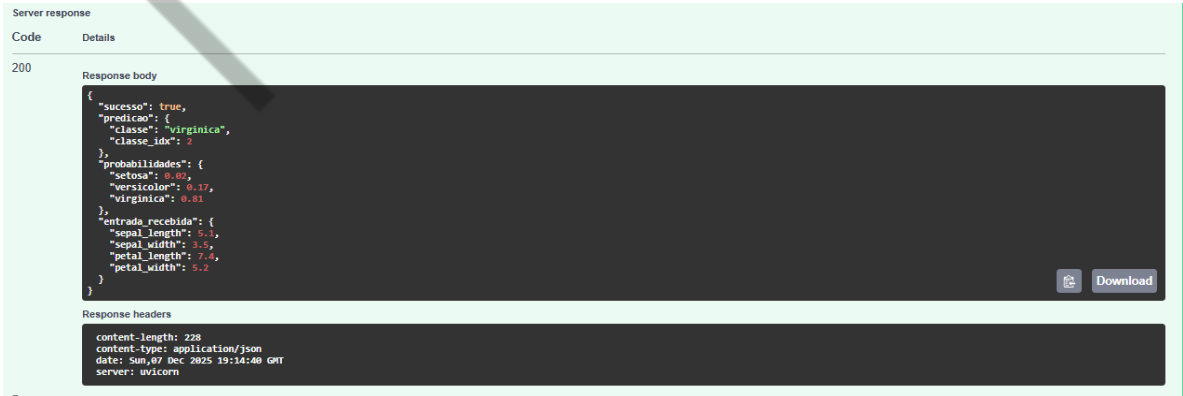


Figura 17 – FastAPI-IRIS-11
Fonte: Elaborado pelo autor (2026)

Agora, tente enviar dados inválidos e veja a mágica do Pydantic:

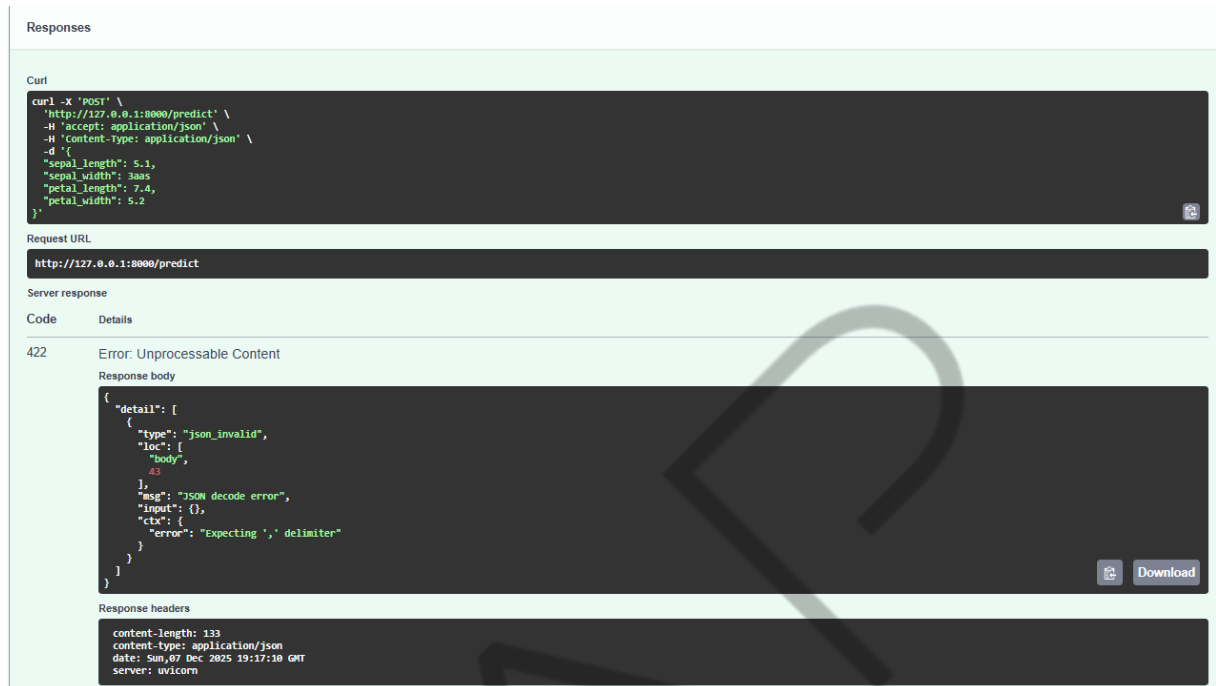


Figura 18 – FastAPI-IRIS-12
Fonte: Elaborado pelo autor (2026)

E, por fim, observe a documentação de alto nível que temos com o ReDoc:

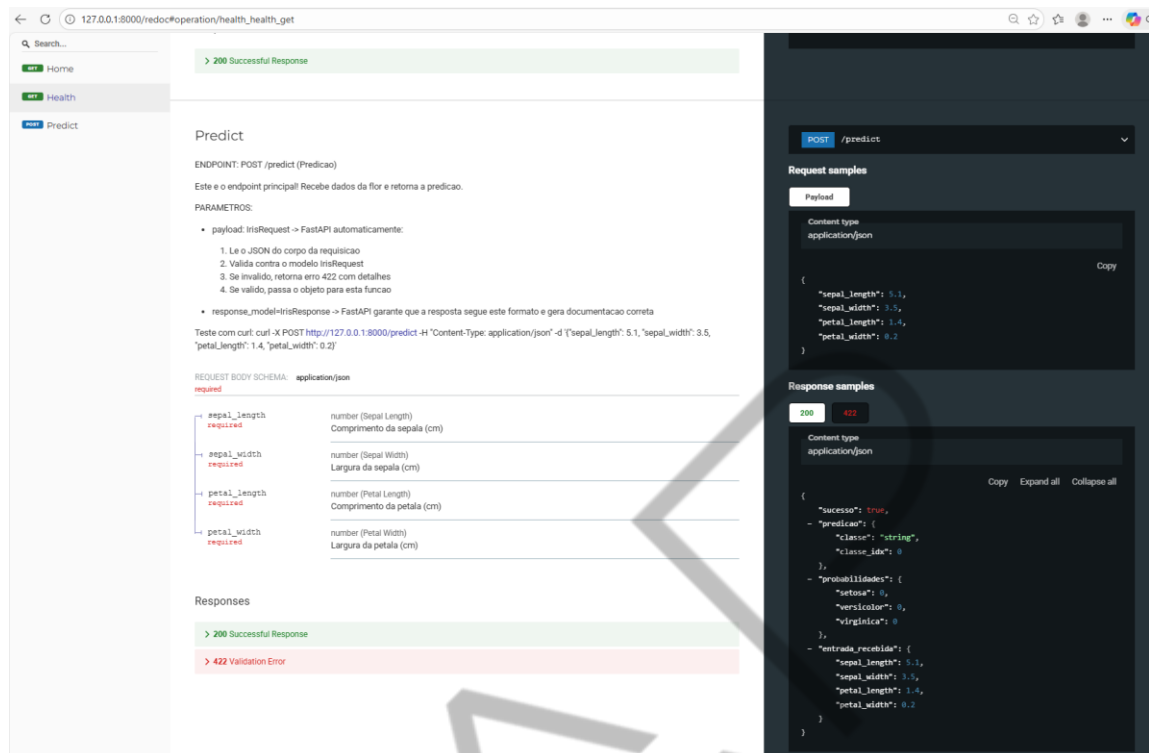


Figura 19 – FastAPI-IRIS-13
Fonte: Elaborado pelo autor (2026)

Ao comparar com o Flask, é impossível não perceber o poder da FastAPI: antes precisaríamos escrever diversas linhas de código só para validação, se o JSON existe, campos obrigatórios, verificar tipos, retornar erros apropriados... No FastAPI com Pydantic: 0 linhas de validação manual. Tudo é declarativo!

Endpoints síncronos vs assíncronos (def vs async def)

O FastAPI tem suporte nativo a assincronicidade com `async def`. Isso significa que, se você tiver operações IO-bound (chamadas a banco, outros serviços, leitura de arquivos etc), pode ganhar desempenho atendendo múltiplas requisições em paralelo.

Em resumo:

- IO-bound: o gargalo é espera de I/O (rede, disco etc.). Async pode ajudar muito.

- CPU-bound: o gargalo é processamento pesado (ex.: modelo de ML muito grande). Aí, o ganho vem mais de múltiplos workers/processos do que async.

Vamos ver um exemplo simples com um endpoint síncrono e um assíncrono.

```
app = FastAPI(title="Exemplo FastAPI Async")
```

```
@app.get("/sync-wait")
```

```
def sync_wait():
```

```
    """Espera 2 segundos de forma bloqueante (síncrona)."""
```

```
    import time
```

```
    time.sleep(2)
```

```
    return {"message": "Resposta síncrona depois de 2 segundos"}
```

```
@app.get("/async-wait")
```

```
async def async_wait():
```

```
    """Espera 2 segundos de forma assíncrona.
```

```
    Enquanto isso, o servidor pode atender outras requisições.
```

```
    """
```

```
    await asyncio.sleep(2)
```

```
    return {"message": "Resposta assíncrona depois de 2 segundos"}
```

Quando async ajuda (e quando não)

Se seu endpoint faz chamadas a serviços externos, consulta bancos de dados, lê arquivos e essas bibliotecas suportam async, vale muito a pena usar async def.

Se o seu endpoint só faz um cálculo pesado em CPU como a inferência de um modelo grande sem I/O, o async não vai trazer grande benefício. Nesse caso, o caminho é rodar o servidor com mais workers:

```
uvicorn app_fastapi_predict:app --workers 4
```

Middlewares no FastAPI

Um middleware é um componente que intercepta a requisição antes de chegar no endpoint e/ou a resposta antes de voltar para o cliente. Você pode usar middlewares para logar tempo de resposta, adicionar headers, fazer autenticação, CORS etc.

Vamos criar um exemplo simples de middleware que adiciona um header X-Process-Time com o tempo de processamento da requisição:

```
import time

from fastapi import FastAPI, Request

app = FastAPI(title="Exemplo de Middleware no FastAPI")

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time
```

```
response.headers["X-Process-Time"] = str(process_time)

return response
```

```
@app.get("/")
```

```
def read_root():
```

```
    return {"message": "Hello with middleware"}
```

No terminal:

```
python -m uvicorn app_fastapi_middleware:app --reload
```

Ao rodar este app e fazer uma requisição, você verá na resposta HTTP um header extra: - `X-Process-Time: <algum valor em segundos>`

Conseguimos ver com F12 no navegador a partir das ferramentas de desenvolvedor ou pelo Postman, onde fica ainda mais fácil: após fazer uma requisição GET para `http://127.9.9.1:8000/`, na resposta clique na aba Headers e procure por `x-process-time`:

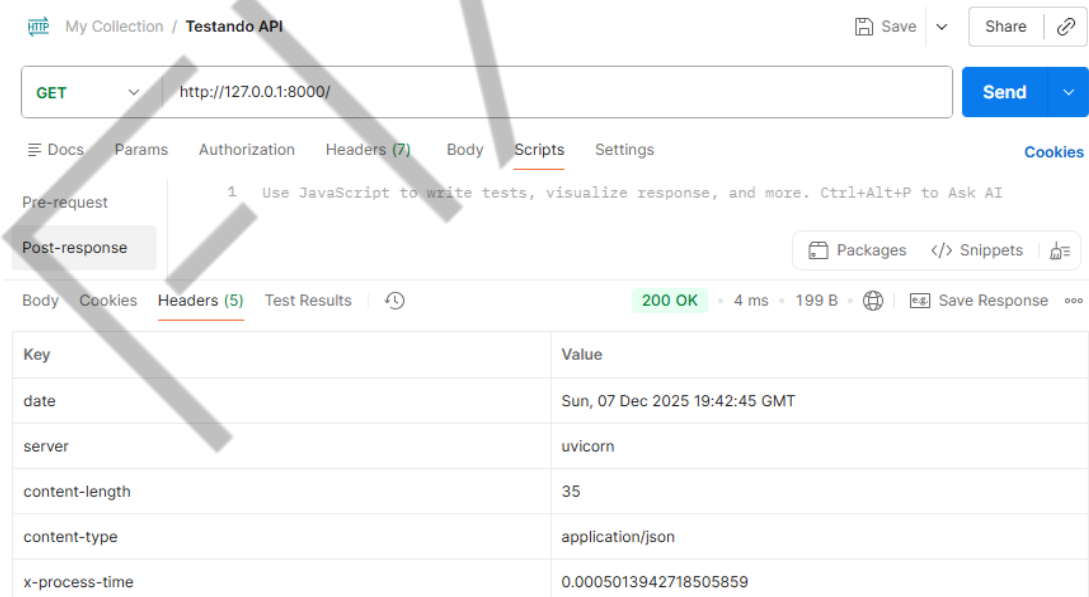


Figura 20 – FastAPI-Postman x-process-time
Fonte: Elaborado pelo autor (2026)

Dependency Injection e Carregamento do Modelo

O problema: recarregar o modelo a cada requisição. Imagine que seu modelo leva 5 segundos para carregar do disco. Se você carregar a cada requisição, para uma seriam apenas 5 segundos, mas para 100 seriam 500 segundos desperdiçados! A solução: Dependency Injection com Cache! O FastAPI tem um sistema elegante de Dependency Injection que resolve isso:

```
from functools import lru_cache
from fastapi import Depends

@lru_cache # ← Cacheia o resultado!
def get_classifier():
    return IrisClassifier() # Carrega UMA VEZ

@app.post("/predict")
def predict(
    payload: IrisRequest,
    classifier: IrisClassifier = Depends(get_classifier) # ← Injetado!
):
```

Dependency Injection

Requisição 1:

predict(payload) -> Dependência: get_classifier -> Criação do IrisClassifier (carrega o modelo).

Requisições 2, 3, 4...:

predict(payload) -> Dependência: get_classifier -> Retorno do CACHE!
(instantâneo).

```
@lru_cache
def get_classifier() -> IrisClassifier:
    """
    Função de dependência que cria e cacheia o classificador.

    O decorator @lru_cache garante que:
    - O modelo seja carregado apenas UMA VEZ
    - Todas as requisições compartilhem a mesma instância
    - Não há overhead de recarregar o modelo

    Em produção, isso é CRÍTICO para performance!
    """
    return IrisClassifier()
```

Figura 21 – FastAPI-Dependency Injection
Fonte: Elaborado pelo autor (2026)

VANTAGEM	DESCRIÇÃO
Performance	Modelo carregado 1x, não 1000x
Organização	Separação clara de responsabilidades
Testabilidade	Fácil substituir por mocks
Escalabilidade	Cada worker tem sua instância cacheada

Tabela 3 - Vantagens do padrão Dependency Injection
Fonte: Elaborado pelo autor (2026)

Predição em Lote (Batch)

Vamos aproveitar e adicionar um endpoint `/predict-batch` que aceita múltiplas flores no json:

```
{
  "amostras": [
    {"sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4, "petal_width":
0.2},
    {"sepal_length": 6.3, "sepal_width": 3.3, "petal_length": 6.0, "petal_width":
2.5}
  ]
}
```

```
@app.post("/predict-batch", response_model=IrisBatchResponse, tags=["Predição"])
def predict_batch(
    payload: IrisBatchRequest,
    classifier: IrisClassifier = Depends(get_classifier)
):
    """
    Predição para múltiplas flores de uma vez (batch).

    Mais eficiente do que fazer N requisições separadas!
    """
    # Converter lista de requests para array numpy
    features = np.array([
        [a.sepal_length, a.sepal_width, a.petal_length, a.petal_width]
        for a in payload.amostras
    ])

    predictions = classifier.predict_batch(features)

    return IrisBatchResponse(
        total_amostras=len(predictions),
        predicoes=predictions
    )
```

Figura 22 – FastAPI-Predict-Batch
Fonte: Elaborado pelo autor (2026)

Considerações Finais

O FastAPI representa o estado da arte para o desenvolvimento de APIs de Machine Learning em Python. Não é só “porque está na moda”. Ele resolve, de forma concreta, vários problemas que você teria que ficar remendando no Flask:

- Você escreve menos código para chegar no mesmo resultado e a validação automática com Pydantic transforma aquele caos de if campo not in body em modelos tipados, com erros claros e previsíveis.
- A documentação interativa vem de graça: /docs e /redoc já mostram seus endpoints, modelos e exemplos. Não tem aquele ritual de “um dia eu documento”, que na prática nunca chega.
- Em termos de performance, ele não fica devendo: roda em cima de um stack ASGI moderno (Starlette + Uvicorn), entregando throughput digno de linguagem compilada para a maioria dos cenários de API de ML.

Se você quiser ir além, os próximos passos naturais são mergulhar na documentação oficial do FastAPI, do Pydantic, do Starlette e do Uvicorn: essas são as peças que, juntas, sustentam essa experiência mais moderna de APIs para Machine Learning.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, evoluímos do Flask para o FastAPI, um framework moderno pensado justamente para APIs de alto desempenho em Python. Vimos como criar uma aplicação FastAPI desde o zero, definir e explorar endpoints na prática e a documentação automática em /docs, que já nasce integrada com os modelos de entrada e saída.

Você aprendeu a usar Pydantic para descrever e validar os dados da requisição, substituindo if manuais por modelos tipados e mais seguros. Reimplementamos a rota /predict que já existia em Flask, agora com tipagem explícita, validação automática e respostas estruturadas, além de discutirmos rapidamente onde entram os endpoints async e quando isso faz diferença em APIs de ML.

Não se esqueça de revisar o código da aula, explorar a documentação oficial do FastAPI e brincar com seus próprios modelos usando Pydantic: é assim que essa “mágica” de tipos, validação e docs automáticas começa a ficar natural no seu dia a dia.

REFERÊNCIAS

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps: Operationalizing Machine Learning Models**. Sebastopol: O'Reilly Media, 2021.

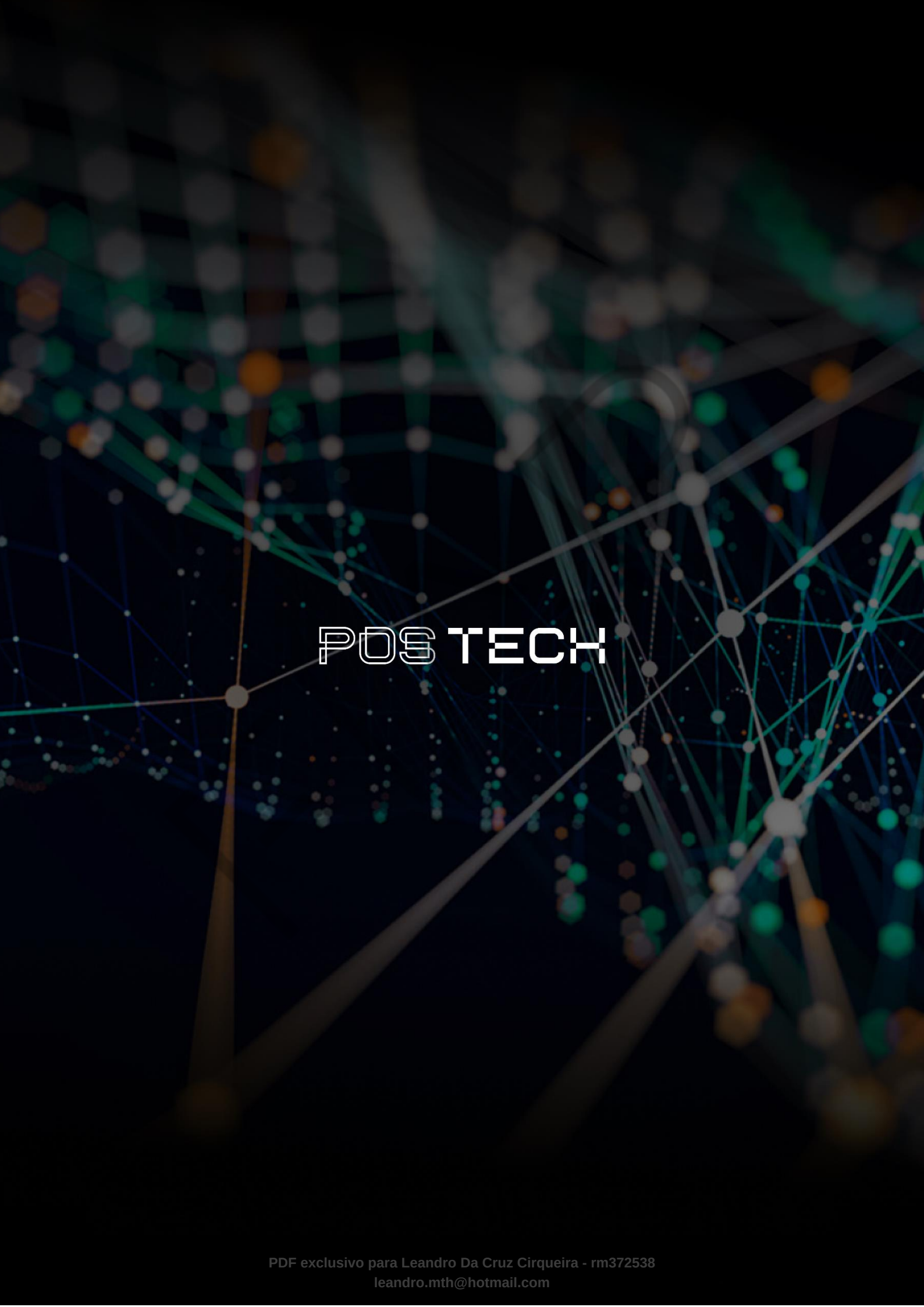
LUBANOVIC, B. **FastAPI: Modern Python Web Development**. Sebastopol: O'Reilly Media, 2023.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2021.

PALAVRAS-CHAVE

FastAPI. API. WEB HTTP. REST. API RESTful. Pydantic. Model. Batch.

EMSE



POSTECH